

ARTI406 – Assignment 2 Report: Data Quality Assessment & Preprocessing

Student Info:

- Name: Alzhra Abdullah Al Nasser
- Student ID: 2230005493
- Dataset Used: Fastfood Dataset (fastfood_cleaned.csv)
- GitHub Repository Link: <https://github.com/Alzhraa04/ML-Assignment1>

Task 1: Identify Data Quality Issues:

In the baseline dataset, the data types are already correctly structured as numerical (int64/float64) and categorical (object). A complete missing value scan showed 0 missing values. To demonstrate data quality remediation according to the assignment guidelines, artificial missing values (NaN) were introduced into the first 6 rows of the calories column.

```
df.isna().sum()
restaurant 0
item 0
calories 0
cal_fat 0
total_fat 0
sat_fat 0
trans_fat 0
cholesterol 0
sodium 0
total_carb 0
fiber 0
sugar 0
protein 0
vit_a 0
vit_c 0
calcium 0
saled 0
dtype: int64
```

	restaurant	item	calories	cal_fat	total_fat	sat_fat	trans_fat	cholesterol
0	Mcdonalds	Artisan Grilled Chicken Sandwich	NaN	60	7	2.0	0.0	
1	Mcdonalds	Single Bacon Smokehouse Burger	NaN	410	45	17.0	1.5	
2	Mcdonalds	Double Bacon Smokehouse Burger	NaN	600	67	27.0	3.0	
3	Mcdonalds	Grilled Bacon Smokehouse Chicken Sandwich	NaN	280	31	10.0	0.5	
4	Mcdonalds	Crispy Bacon Smokehouse Chicken Sandwich	NaN	410	45	12.0	0.5	
5	Mcdonalds	Big Mac	NaN	250	28	10.0	1.0	
6	Mcdonalds	Cheeseburger	300.0	100	12	5.0	0.5	
7	Mcdonalds	Classic Chicken Sandwich	510.0	210	24	4.0	0.0	
8	Mcdonalds	Double Cheeseburger	430.0	190	21	11.0	1.0	
9	Mcdonalds	Double Quarter Pounder® with Cheese	770.0	400	45	21.0	2.5	

Task 2: Apply One Missing Value Strategy:

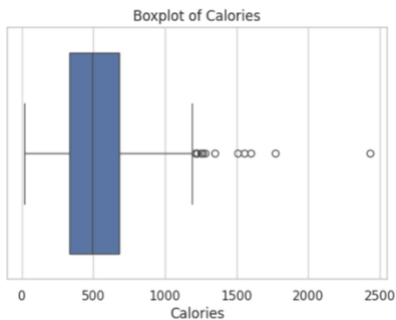
Mean Imputation was applied to resolve the missing values. Deleting the rows would shrink the dataset and completely remove unique menu items (like the Big Mac). Mean imputation preserves all 513 original rows and maintains statistical consistency with the aggregate calculations established in Assignment 1. The calculated mean of “528.40 calories” was filled into the empty cells.

```
Calculated Mean Value used for filling: 528.40 calories
Missing values remaining in 'calories' column: 0

--- Verified First 10 Rows of Imputed Dataset ---
restaurant      item      calories
0 Mcdonalds      Artisan Grilled Chicken Sandwich  528.402367
1 Mcdonalds      Single Bacon Smokehouse Burger    528.402367
2 Mcdonalds      Double Bacon Smokehouse Burger    528.402367
3 Mcdonalds      Grilled Bacon Smokehouse Chicken Sandwich  528.402367
4 Mcdonalds      Crispy Bacon Smokehouse Chicken Sandwich  528.402367
5 Mcdonalds      Big Mac      528.402367
6 Mcdonalds      Cheeseburger  300.000000
7 Mcdonalds      Classic Chicken Sandwich  510.000000
8 Mcdonalds      Double Cheeseburger  430.000000
9 Mcdonalds      Double Quarter Pounder® with Cheese  770.000000
```

Task 3: Detect and Handle Outliers Using IQR:

Using the Interquartile Range (IQR) formula, the upper outlier boundary was determined to be “1205.0 calories”. A total of (12 outlier items) were detected sitting above this threshold. Instead of dropping them (which would delete large family platters and promotional items), a Percentile Capping Method was applied. Extreme values were clipped at the 95th percentile, reducing the absolute maximum calorie count from a skewed 2430.0 down to a controlled “1030.0 calories”.



```
lower_cap = df['calories'].quantile(0.05)
upper_cap = df['calories'].quantile(0.95)

# Create a copy and apply the clip operation
df_capped = df.copy()
df_capped['calories'] = df_capped['calories'].clip(lower_cap, upper_cap)

print(f"Values below 5th percentile capped at: {lower_cap} calories")
print(f"Values above 95th percentile capped at: {upper_cap} calories")
print(f"Original Maximum Calories: {df['calories'].max()}")
print(f"New Maximum Calories after Capping: {df_capped['calories'].max()}")

# Overwrite our primary dataframe state to carry clean features forward
df = df_capped.copy()

Values below 5th percentile capped at: 170.0 calories
Values above 95th percentile capped at: 1030.0 calories
Original Maximum Calories: 2430.0
New Maximum Calories after Capping: 1030.0
```

```
# Calculate IQR Boundaries
Q1 = df['calories'].quantile(0.25)
Q3 = df['calories'].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Identify Outliers
outliers = df[(df['calories'] < lower_bound) | (df['calories'] > upper_bound)]

print(f"Lower Bound Boundary: {lower_bound}")
print(f"Upper Bound Boundary: {upper_bound}")
print(f"Total Outliers Detected: {len(outliers)} items")

... Lower Bound Boundary: -195.0
Upper Bound Boundary: 1205.0
Total Outliers Detected: 12 items
```

Task 4: Normalize Numerical Features Using Both Min-Max and Z-Score:

- **Min-Max Normalization:** Rescaled the calories and sodium columns into a strict, bounded range between 0 and 1. This is crucial for distance-based models so that features with massive native scales (like sodium) do not overpower smaller scales.
- **Z-Score Standardization:** Transformed the data to center around a mean of 0 with a standard deviation of 1. This ensures features are equally scaled, which is a structural prerequisite for gradient descent and dimensionality reduction via PCA.

```
from sklearn.preprocessing import MinMaxScaler
```

```
# Select two numerical features for demonstration
features_to_scale = ['calories', 'sodium']
print("---- Baseline Original Features ----")
print(df[features_to_scale].head(5))
```

```
---- Baseline Original Features ----
   calories  sodium
0  528.402367  1110
1  528.402367  1500
2  528.402367  1920
3  528.402367  1940
4  528.402367  1980
```

```
# Initialize and apply Min-Max Scaler
min_max_scaler = MinMaxScaler()
df_scaled = df[features_to_scale].copy()
df_scaled[features_to_scale] = min_max_scaler.fit_transform(df_scaled)

print("\n---- After Min-Max Normalization (Scaled 0 to 1) ----")
print(df_scaled.head(5))
```

```
---- After Min-Max Normalization (Scaled 0 to 1) ----
   calories  sodium
0  0.416747  0.180544
1  0.416747  0.250838
2  0.416747  0.314097
3  0.416747  0.317395
4  0.416747  0.323990
```

```
from sklearn.preprocessing import StandardScaler
# Initialize and apply Standard Scaler
z_scaler = StandardScaler()
df_standardized = df[['calories', 'sodium']].copy()
df_standardized[['calories', 'sodium']] = z_scaler.fit_transform(df_standardized)

print("After Z-Score Standardization (Mean = 0, Std = 1)")
print(df_standardized.head(5))
```

```
After Z-Score Standardization (Mean = 0, Std = 1)
   calories  sodium
0  0.042474 -0.198860
1  0.042474  0.401022
2  0.042474  0.574238
3  0.042474  1.003195
4  0.042474  1.061126
```

```
# Verifying the mean is 0 and standard deviation is 1
print("\n Verification Statistics:")
print("Means (Should be close to 0):")
print(df_standardized.mean().round(4))
print("\nStandard Deviations (Should be exactly 1):")
print(df_standardized.std().round(4))
```

```
... Verification Statistics
Means (Should be close to 0):
calories    0.0
sodium     -0.0
dtype: float64

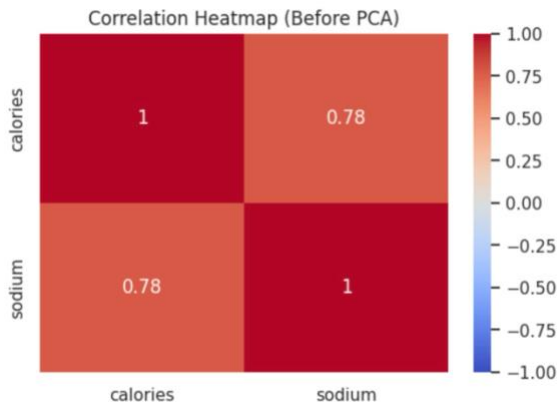
Standard Deviations (Should be exactly 1):
calories    1.001
sodium      1.001
dtype: float64
```

Task 5: Apply PCA and Interpret Explained Variance:

A correlation check before PCA revealed a strong positive correlation of 0.78 between calories and sodium, making the dataset an ideal candidate for dimensionality reduction. Principal Component Analysis (PCA) successfully compressed these features:

- Principal Component 1 (PC1) captures 88.76% of the entire variance.
- Principal Component 2 (PC2) captures the remaining 11.24%.

Since PC1 captures nearly 89% of the total dataset variation alone, we can effectively drop PC2 and represent our data using a single dimension with minimal information loss.



```
from sklearn.decomposition import PCA

# Prepare features matrix
X = df_standardized[['calories', 'sodium']]

# Apply PCA for 2 components
pca = PCA(n_components=2)
principal_components = pca.fit_transform(X)

print(" PCA RESULTS ")
print("Explained Variance Ratio per component:", pca.explained_variance_ratio_)
print(f"Total Explained Variance by PC1 alone: {pca.explained_variance_ratio_[0]*100:.2f}%")
print(f"Total Combined Variance (PC1 + PC2): {sum(pca.explained_variance_ratio_)*100:.2f}%")
```

PCA RESULTS
Explained Variance Ratio per component: [0.8876202 0.1123798]
Total Explained Variance by PC1 alone: 88.76%
Total Combined Variance (PC1 + PC2): 100.00%

